

MP2: Simple File System

Isaac Burns

UIN: 935007106

CSCE410: Operating Systems

Assigned Tasks

Main: Completed.

System Design

The goal of machine problem 2 was to create a contiguous frame manager, capable of three actions:

1. Freeing contiguous blocks of frames
2. Marking contiguous blocks of frames inaccessible
3. Allocating contiguous blocks of frames

Please note that each frame is itself a contiguous block of memory as well.

For each of the above functions, the frame manager is split into frame pools. You can only allocate or mark inaccessible, blocks of memory owned by that specific frame pool. However, freeing memory can be done from the viewpoint of the frame manager, which itself checks each frame pool to find the owner of that block.

Internally, each frame pool keeps track of its frames through a bitmap, which utilizes relative positioning. Frame 0 of the bitmap is in fact *base_frame_no* of the pool.

Code Description

I compiled my code through the TA-provided makefile and ran it through *qemu - system - x86_64 - kernelkernel.bin*

cont.frame_pool.C: get_state Assumes FrameState requires 2 bits of space. Indexes into the bitmap using relative frame numbers - fno 0 is bitmap[0], regardless of the base frame no of the pool. Returns FrameState at bitmap[fno].

```
ContFramePool::FrameState ContFramePool::get_state(unsigned long _frame_no) {
    unsigned long byte_idx = _frame_no >> 2;          // / 4
    unsigned char shift    = (_frame_no & 0x3) << 1;  // (% 4) * 2
    unsigned char state    = (bitmap[byte_idx] >> shift) & 0x03;
    return static_cast<ContFramePool::FrameState>(state);
}
```

cont.frame_pool.C: set_state Assumes FrameState requires 2 bits of space. Indexes into the bitmap using relative frame numbers - fno 0 is bitmap[0], regardless of the base frame no of the pool. Sets FrameState at bitmap[fno] to state.

```
void ContFramePool::set_state(unsigned long _frame_no, FrameState _state) {
    unsigned long byte_idx = _frame_no >> 2;
    unsigned char shift    = (_frame_no & 0x3) << 1;
    unsigned char mask     = 0x03 << shift;
    bitmap[byte_idx] = (bitmap[byte_idx] & ~mask)
        | ((static_cast<unsigned char>(_state) & 0x03) << shift);
}
```

cont.frame_pool.C: ContFramePool (constructor) Saves all of the given arguments into local vars, and calculates info frames needed. Checks if info frames are managed by itself; If they are, info frames start at base frame no. Sets all frames to free, and then recaptures info frames as used. The frame manager is the statically defined linked list of ContFramePool pointers. Within that linked list, it sets its own next to 0, and if there is no head, it serves as one. If there is one, it finds the first ContFramePool with no next, and serves as its next.

```
ContFramePool* ContFramePool::head = 0;
ContFramePool::ContFramePool(unsigned long _base_frame_no,
                              unsigned long _n_frames,
                              unsigned long _info_frame_no)
{
    base_frame_no = _base_frame_no;
    info_frame_no = _info_frame_no;
    nFreeFrames = _n_frames;
    nFrames = _n_frames;
    nInfoFrames = needed_info_frames(nFrames);
    if(_info_frame_no == 0){
        bitmap = (unsigned char*) (_base_frame_no * FRAME_SIZE);
    }else{
        bitmap = (unsigned char*) (_info_frame_no * FRAME_SIZE);
    }
    for(unsigned long fno = 0; fno < nFrames; fno++){
        set_state(fno, FrameState::Free);
    }
    if(_info_frame_no == 0){
        set_state(0, FrameState::HoS);
        for(unsigned long fno = 1; fno < nInfoFrames; fno++){
            set_state(fno, FrameState::Used);
        }
    }
}
```

```

    }
    nFreeFrames -= nInfoFrames;;
}
next = 0;
if(head == 0) head = this;
else{
    ContFramePool* p = head;
    while(p->next){
        p = p->next;
    }
    p->next = this;
};
Console::puts("ContframePool initialized");
}

```

cont_frame_pool.C: get_frames Rejects invalid arguments. Loops until it finds a block of free frames of the requested size. It sets the first frame in that block as HoS, and the rest as Used. It decrements nFreeFrames by the requested block size, and returns the absolute frame number of the start of the allocated block.

```

unsigned long ContFramePool::get_frames(unsigned int _n_frames)
{
    if(_n_frames == 0 || _n_frames > nFreeFrames) return 0;
    unsigned long l = 0;
    for(unsigned long r = 0; r < nFrames; r++){
        if(get_state(r) != FrameState::Free){
            l = r + 1;
            continue;
        }
        if(r - l + 1 == _n_frames){
            set_state(l, FrameState::HoS);
            for(unsigned long i = l+1; i <= r; i++){
                set_state(i, FrameState::Used);
            }
            nFreeFrames -= _n_frames;
            return base_frame_no + l;
        }
    }
    return 0;
}

```

cont_frame_pool.C: mark_inaccessible It checks that the requested base frame no is after the base frame no of the executing frame pool, and rejects if that is not the case. It then clamps the absolute limit to the last frame of the executing frame pool. It then sets all frames between requested base to the clamped limit to Inaccessible, updating the free count as it goes.

```

void ContFramePool::mark_inaccessible(unsigned long _base_frame_no,
                                     unsigned long _n_frames)
{
    if(_base_frame_no < base_frame_no){
        Console::puts("Frame pool ... was requested to mark frame ... inaccessible, but the frame pool ...");
        return;
    }
    unsigned long limit_request = _base_frame_no + _n_frames;
    unsigned long limit_pool = base_frame_no + nFrames;
    unsigned long limit = limit_request > limit_pool ? limit_pool : limit_request;
    unsigned long limit_bitmap = limit - base_frame_no;
}

```

```

    unsigned long start = _base_frame_no - base_frame_no;
    for(unsigned long i = 0; i < limit_bitmap; i++){
        if(get_state(i) == FrameState::Free) nFreeFrames--;
        set_state(i, FrameState::Inaccessible); //assumption is this will just overwrite anything
    }
}

```

cont_frame_pool.C: release_frames Finds the first frame pool that owns the given first frame no, and delegates functionality to release frames internal, executed through that frame pool.

```

void ContFramePool::release_frames(unsigned long _first_frame_no)
{
    for(ContFramePool* p = head; p; p = p->next){
        if(_first_frame_no >= p->base_frame_no && _first_frame_no < p->base_frame_no + p->nFrames){
            p->release_frames_internal(_first_frame_no);
            return;
        }
    }
    Console::puts("Could not find frame pool for release frames request...\n");
}

```

cont_frame_pool.C: release_frames_internal First frame no is absolute, so it shifts it to the relative index within the bitmap. It checks that frame in the bitmap is a head of sequence, otherwise it rejects the request. It then iterates and sets blocks free until it hits another Free, Inaccessible, or HoS, updating nFreeFrames along the way.

```

void ContFramePool::release_frames_internal(unsigned long _first_frame_no){
    unsigned long i = _first_frame_no - base_frame_no; //bitmap[i] = _first_frame_no
    if(get_state(i) != FrameState::HoS){
        Console::puts("release_frames: first frame not HoS\n");
        return;
    }
    set_state(i, FrameState::Free);
    nFreeFrames++;
    for(i++; i < nFrames && get_state(i) == FrameState::Used; i++){ //FrameState::HoS or FrameState::I
        set_state(i, FrameState::Free);
        nFreeFrames++;
    }
}

```

cont_frame_pool.C: needed_info_frames Calculates the frames that fit into each info frame as $(FRAME_SIZE * 8) / bits_per_frame$; It then returns the ceil of n frames over frames per info frame.

```

unsigned long ContFramePool::needed_info_frames(unsigned long _n_frames)
{
    unsigned long bits_per_frame = 2;
    unsigned long frames_per_info_frame = (FRAME_SIZE * 8) / bits_per_frame; //frame_size is in bytes
    return (_n_frames + frames_per_info_frame - 1) / frames_per_info_frame; //(n + d - 1) / d => ceil
}

```

Testing

I relied on the test provided functions. There are many test cases that were either not tested, or explicitly cause errors. For example, if you mark an allocated block as inaccessible, the kernel does not warn it, it simply does it and assumes you understand the ramifications. When you are releasing a

block of memory, the first frame must be the start of the sequence, and it is rejected otherwise. When you are attempting to release frames, it assumes that the first frame pool it finds that owns the frame is the correct one.